实践项目1：使用 GitHub Actions 实现自动化测试

任务目标

通过本任务，你将学会：

1. 创建基本的GitHub Actions工作流配置文件
2. 触发工作流运行并观察其执行过程
3. 验证工作流是否成功执行

任务要求

你需要完成以下核心操作：

1. 创建一个GitHub仓库
2. 准备一个python项目
3. 创建并编写工作流配置文件 (ci.yml)
4. 推送代码，触发工作流并验证测试结果
5. 在头歌平台进行测评

步骤1 - 创建仓库

1. 登录你的GitHub账号
2. 点击 New repository (新建仓库)
3. 设置仓库信息：
   - 仓库名: github-actions-practice (可自定义)
   - 公开性: Public (必须公开！)
   - 描述: 我的第一个GitHub Actions实践
4. 点击 Create repository

步骤2 -准备Python项目

创建一个标准的Python项目结构：

```
my-python-project/
├── .github/workflows/  # 工作流文件目录
│   └── ci.yml
├── calculator.py       # 被测试的代码
├── test_calculator.py  # 测试代码
└── requirements.txt    # 项目依赖（可选）
```

步骤2 -准备Python项目

示例代码内容

calculator.py (被测试的功能模块)

```
def add(a, b):
    return a + b
def multiply(a, b):
    return a * b
```

test_calculator.py (单元测试)

```
import pytest
from calculator import add, multiply
def test_add():
    assert add(2, 3) == 5
    assert add(-1, 1) == 0
def test_multiply():
    assert multiply(3, 4) == 12
    assert multiply(2, 0) == 0
```

步骤3 -创建并配置工作流

在 .github/workflows/ci.yml 文件中配置：

```yaml
name: Python单元测试
on: [push, pull_request] # 推送或提PR时触发
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
    - name: 检出代      码
      uses: actions/checkout@v4
    - name: 设置Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.11'
    - name: 安装依赖
      run: |
        python -m pip install --upgrade pip
        pip install pytest
    - name: 运行测试
      run: pytest -v
```

工作流步骤解析

1. 检出代码：获取仓库代码到虚拟环境
2. 设置Python：配置指定版本的Python环境
3. 安装依赖：安装测试所需的包（pytest）
4. 运行测试：执行pytest命令，-v参数显示详细结果

步骤4 -触发并验证工作流

1. 将所有文件推送到GitHub仓库
2. 点击顶部的 Actions 标签页
3. 选择 Python单元测试 工作流
4. 查看运行状态和详细日志

如何判断测试结果?

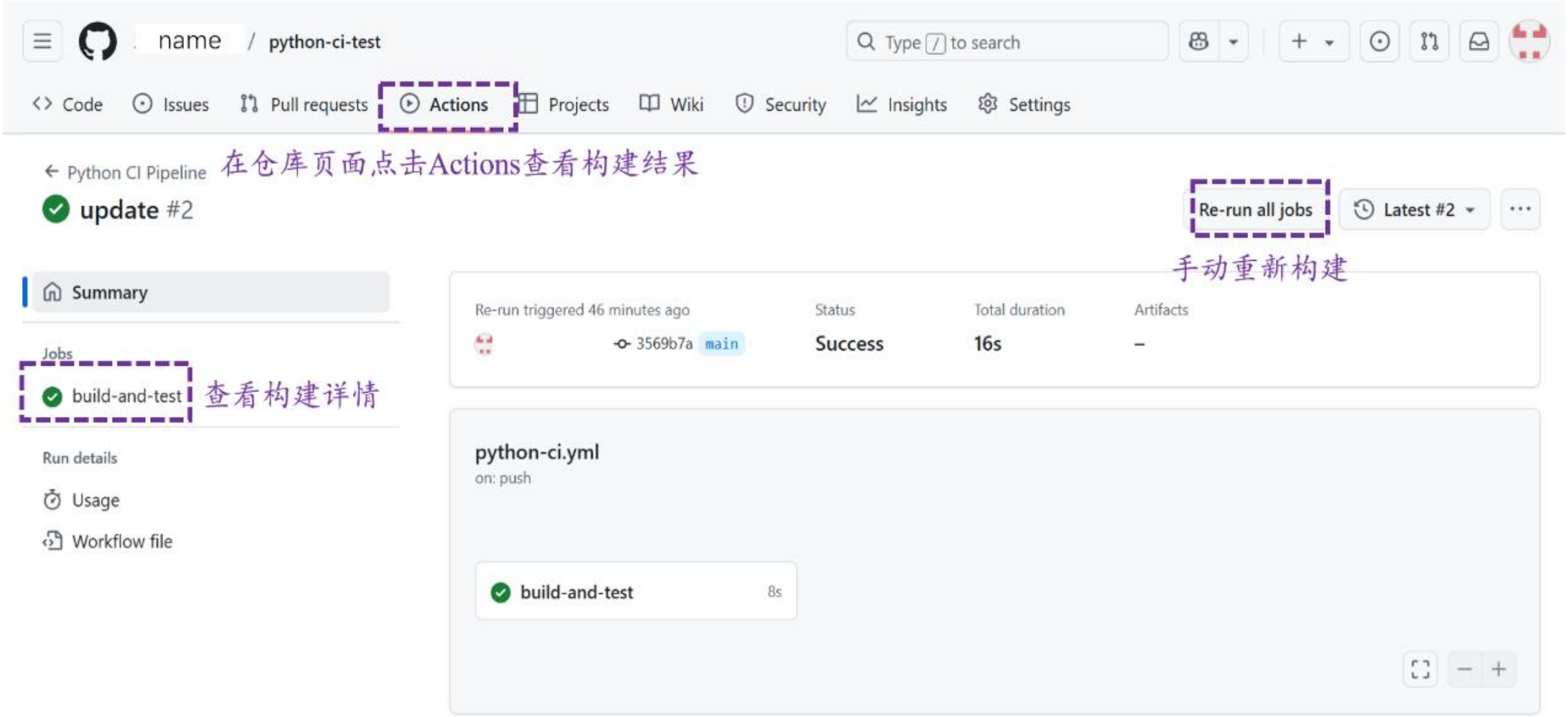
测试通过(成功):

状态为绿色对勾
日志中显示：===== PASSED =====
看到所有测试用例通过的信息

测试失败:

状态为红色叉号
日志中显示：===== FAILED =====
明确指出哪个测试用例失败及原因

结果展示如图


name / python-ci-test
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
在仓库页面点击Actions查看构建结果
Python CI Pipeline
update #2
Re-run all jobs
Latest #2
手动重新构建
Summary
Jobs
build-and-test
查看构建详情
Run details
Usage
Workflow file
Re-run triggered 46 minutes ago
3569b7a main
Status
Success
Total duration
16s
Artifacts
–
python-ci.yml
on: push
build-and-test
8s

步骤5 -尝试一个失败的案例

1. 修改 calculator.py 中的 add 函数，使其返回错误结果

```
def add(a, b):
    return a * b  # 错误的实现
!
```

2. 提交并推送代码
3. 观察工作流运行状态变为失败
4. 查看日志，学习如何阅读错误信息

步骤6 -提交任务

1. 确保最后一次工作流运行成功（绿色对勾）
2. 复制你的仓库地址：
   https://github.com/你的用户名/你的仓库名
3. 在头歌平台提交该链接



实践项目2：通过GitHub Actions自动部署静态文档到GitHub Pages

任务目标

本次实践项目旨在让大家掌握使用 GitHub Actions 实现静态网站（MkDocs 文档）的自动化构建与部署，最终通过 GitHub Pages 托管可访问的文档网站，具体目标如下：

1. 搭建 MkDocs 文档项目结构
2. 配置 GitHub Actions 工作流实现自动部署
3. 验证部署结果，确保文档网站可访问

任务步骤总览

本次实践任务共分为 6 个核心步骤，将带领大家从 0 到 1 完成静态文档的自动化部署：

1. 创建 GitHub 仓库并初始化项目
2. 编写文档内容
3. 配置 MkDocs
4. 配置 GitHub Actions 自动部署工作流
5. 配置仓库并触发部署
6. 提交任务

步骤 1 - 创建 GitHub 仓库并初始化项目

1.1 新建 GitHub 仓库

1.2 克隆仓库到本地并创建目录结构

1. 克隆仓库：使用git clone https://github.com/你的GitHub用户名/你的仓库名.git命令，将仓库克隆到本地
2. 按以下结构创建文件和目录（必填项需确保存在）：

```
my-docs/
├── docs/ # 存放 Markdown 文档（必填）
│   ├── index.md # 主页（必填）
│   ├── about.md # 关于页面（必填）
│   ├── guide.md # 使用指南（必填）
│   └── api.md # API 参考（必填）
├── .github/ # 工作流配置目录（必填）
│   └── workflows/
│       └── deploy.yml # 自动化部署工作流配置（必填）
├── mkdocs.yml # MkDocs 配置文件（必填）
└── README.md # 项目说明（可选）
```

步骤2-编写文档内容

需为docs目录下的4个Markdown文件填充指定内容，确保文档可正常跳转和展示：

2.1 docs/index.md（主页）

```
# 欢迎使用 MkDocs
这是一个示例文档，使用 **MkDocs** 进行构建。
- [关于](about.md)
- [使用指南](guide.md)
- [API 参考](api.md)
```

作用：文档网站的首页，包含到其他页面的跳转链接

2.2 docs/about.md（关于页面）

```
# 关于本项目
本项目使用 MkDocs 生成技术文档，并通过 GitHub Pages 自动部署。
```

作用：介绍项目的核心信息和技术栈

2.3 docs/guide.md（使用指南）

```
# 使用指南
1. 编写 Markdown 文档
2. 提交代码到 GitHub
3. GitHub Actions 自动构建并部署
```

作用：说明文档项目的使用流程

2.4 docs/api.md（API 参考）

```
# API 参考
这里是 API 参考文档示例。
```

作用：展示 API 相关文档示例（可根据实际需求扩展）

步骤 3 - 配置 MkDocs

3.1 创建并配置 mkdocs.yml 文件
在项目根目录（my-docs/）下创建mkdocs.yml文件，用于指定网站主题、导航等核心配置，内容如下：

```
site_name: 我的技术文档  # 网站名称（将显示在页面标题和导航栏）
theme:
  name: material  # 使用 Material 主题（美观且支持响应式）
nav:  # 网站导航结构（对应 docs 目录下的文件）
  - 首页: index.md
  - 关于: about.md
  - 指南: guide.md
  - API: api.md
```

说明：MkDocs 通过该配置文件识别文档结构和网站样式，material主题需后续安装，无需手动下载

步骤4 - 配置 GitHub Actions 自动部署工作流

4.1 创建 deploy.yml 文件

在.github/workflows/目录下创建deploy.yml文件，定义自动化部署的触发条件、运行环境和执行步骤，内容如下：

```yaml
name: Deploy MkDocs to GitHub Pages  # 工作流名称（自定义）
on:  # 触发条件
  push:
    branches:
      - main  # 仅在 main 分支推送代码时触发
    paths:
      - 'docs/**'  # 仅当 docs 目录下文件变化时触发
      - 'mkdocs.yml'  # 仅当 MkDocs 配置文件变化时触发
jobs:  # 工作流任务
  deploy:
    runs-on: ubuntu-latest  # 运行环境：Ubuntu 最新版
    steps:  # 任务执行步骤
      - name: 检出代码  # 步骤1：拉取GitHub仓库代码
        uses: actions/checkout@v4
      - name: 设置Python环境  # 步骤2：配置Python（MkDocs依赖Python）
        uses: actions/setup-python@v4
        with:
          python-version: '3.9'  # 指定Python版本
      - name: 安装MkDocs及主题  # 步骤3：安装依赖工具
        run: |
          pip install mkdocs-material  # 安装MkDocs和Material主题
      - name: 构建静态文档  # 步骤4：生成静态网站文件
        run: |
          mkdocs build  # 生成的文件存放在项目根目录的 site/ 文件夹
      - name: 部署到GitHub Pages  # 步骤5：将静态文件部署到gh-pages分支
        uses: JamesIves/github-pages-deploy-action@v4
        with:
          branch: gh-pages  # 部署目标分支（GitHub Pages默认读取该分支）
          folder: site      # 待部署的文件目录（即步骤4生成的site目录）
```

步骤 5 - 配置仓库并触发部署

5.1 创建 gh-pages 分支（部署目标分支）

1. 进入 GitHub 仓库页面，点击顶部 “Branches”
2. 在 “Find or create a branch” 输入框中输入gh-pages，点击 “Create branch: gh-pages from 'main'”
3. 说明：该分支无需手动添加内容，后续 GitHub Actions 会自动向该分支推送静态文件

5.2 开启工作流权限（关键步骤）

1. 进入 GitHub 仓库页面，点击顶部 “Settings”
2. 在左侧导航栏中找到并点击 “Actions” → “General”
3. 滚动到 “Workflow permissions” 区域：
   - 选择 “Read and write permissions”（工作流需读写权限以推送文件到 gh-pages 分支）
   - 勾选 “Allow GitHub Actions to create and approve pull requests”
4. 点击 “Save” 保存设置

步骤5-配置仓库并触发部署

5.3 提交代码并触发部署

在本地项目根目录打开终端，执行以下命令提交代码：

```
# 1. 将所有文件添加到暂存区
git add .
# 2. 提交代码（备注信息可自定义）
git commit -m "初始化MkDocs项目并配置GitHub
Actions"
# 3. 推送到GitHub的main分支
git push origin main
```

步骤 5 - 配置仓库并触发部署

5.4 验证部署结果

1. 查看工作流状态：进入 GitHub 仓库→点击顶部“Actions”，查看名为“Deploy MkDocs to GitHub Pages”的工作流是否显示“Success”（绿色对勾）
2. 访问文档网站：工作流成功后，等待 1-2 分钟（GitHub Pages 更新需时间），访问地址：https://你的GitHub用户名.github.io/你的仓库名/
   - 示例：若用户名为“testuser”，仓库名为“my-docs”，则访问地址为 https://testuser.github.io/my-docs/

结果展示如图


我的技术文档
Search
我的技术文档
首页
关于
指南
API
欢迎使用 MkDocs
这是一个示例文档，使用 MkDocs 进行构建。
关于
使用指南
API 参考
Made with Material for MkDocs

步骤6-提交任务（环境配置与评测）

6.1 配置实验环境（避免报错）

打开实验环境终端，执行以下命令安装依赖包

```
# 升级requests、urllib3、chardet
pip install --upgrade requests urllib3 chardet
# 安装pyyaml（用于解析配置文件）
pip install pyyaml
```

6.2 提交评测

在实验平台中，输入以下信息提交任务：

- GitHub账户名
- GitHub Token（需提前准备，确保拥有仓库操作权限）
- 仓库链接（格式：https://github.com/你的GitHub用户名/你的仓库名.git）